

Every file, what it does, and what it **talks to**.

A visual map of the whole D:\Aptech\Techwiz\FanHub codebase — drawn from the actual `require()` calls and `res.render()` calls in the code, not guessed.

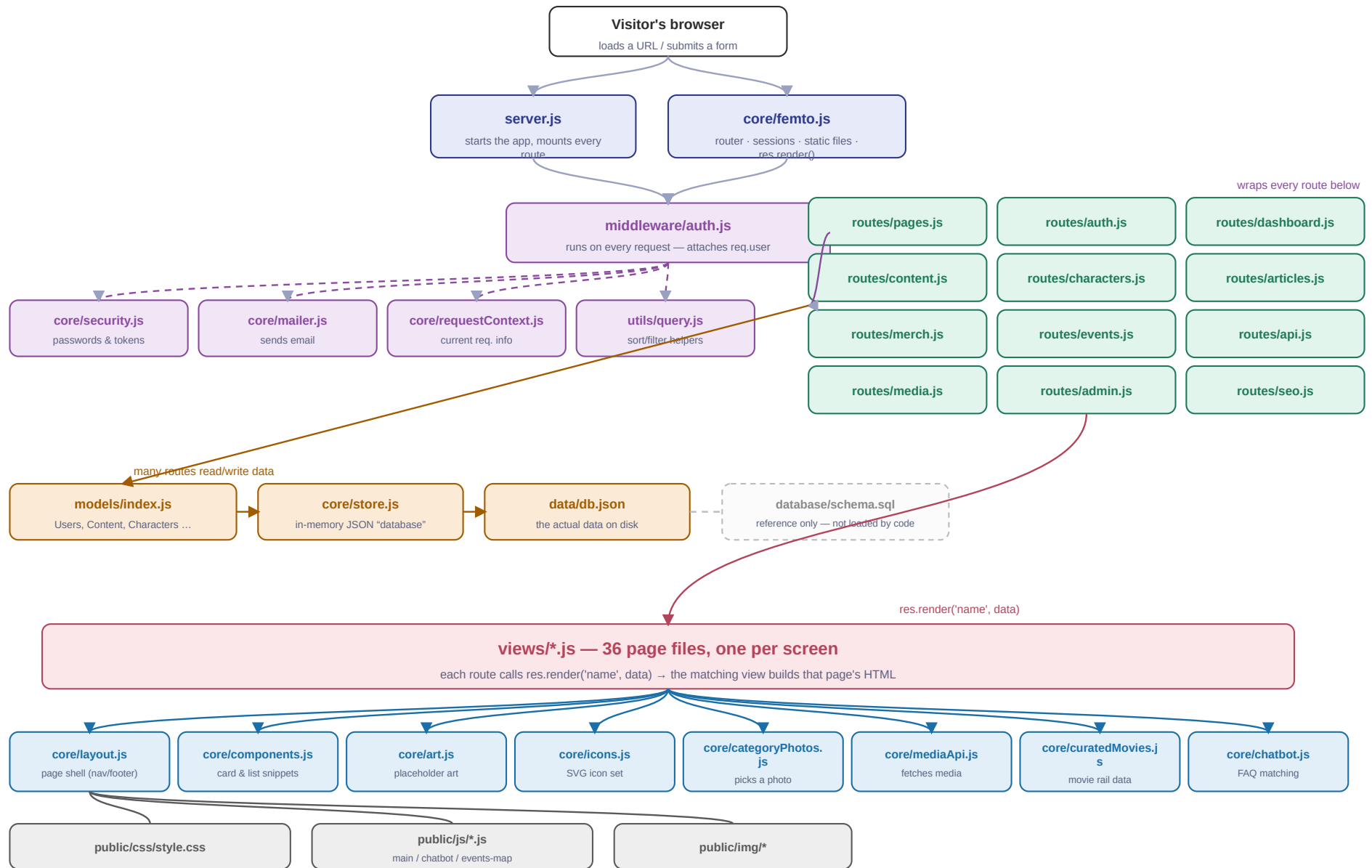
How to read it: each page after this one is a diagram, drawn top-to-bottom or left-to-right in the order a real page request flows through the app. An arrow from box A to box B means **A's code loads B's code** (a `require()`) or **A hands off to B** (a `res.render()` call). Colors group files by role, not by folder.

The last section is a plain reference table — every one of the 72 files in the project, one row each, with a one-line description and exactly which other files it connects to. Use the diagrams to see the shape of the app, and the table to look up any single file.

☐ Server / framework ☐ Middleware / helpers ☐ Routes ☐ Data layer ☐ Views (pages) ☐ Core view-helpers ☐ Static assets

1 • The big picture

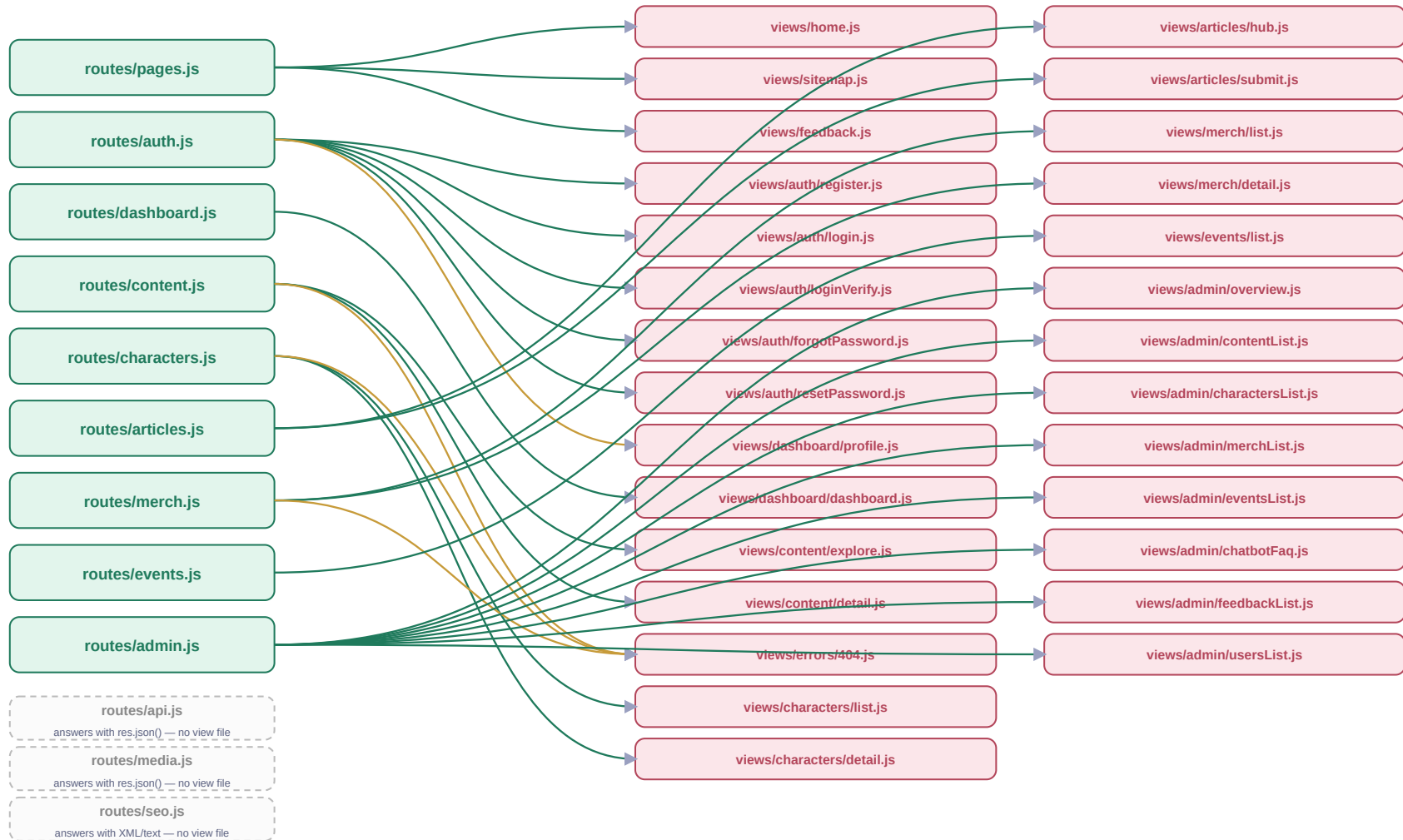
How one page request flows through the whole app, from the browser back to the browser.



Solid arrow = requires / calls that file · dashed = reference or side-use only · colors group files by role

2 • Routes → Views (the exact res.render() map)

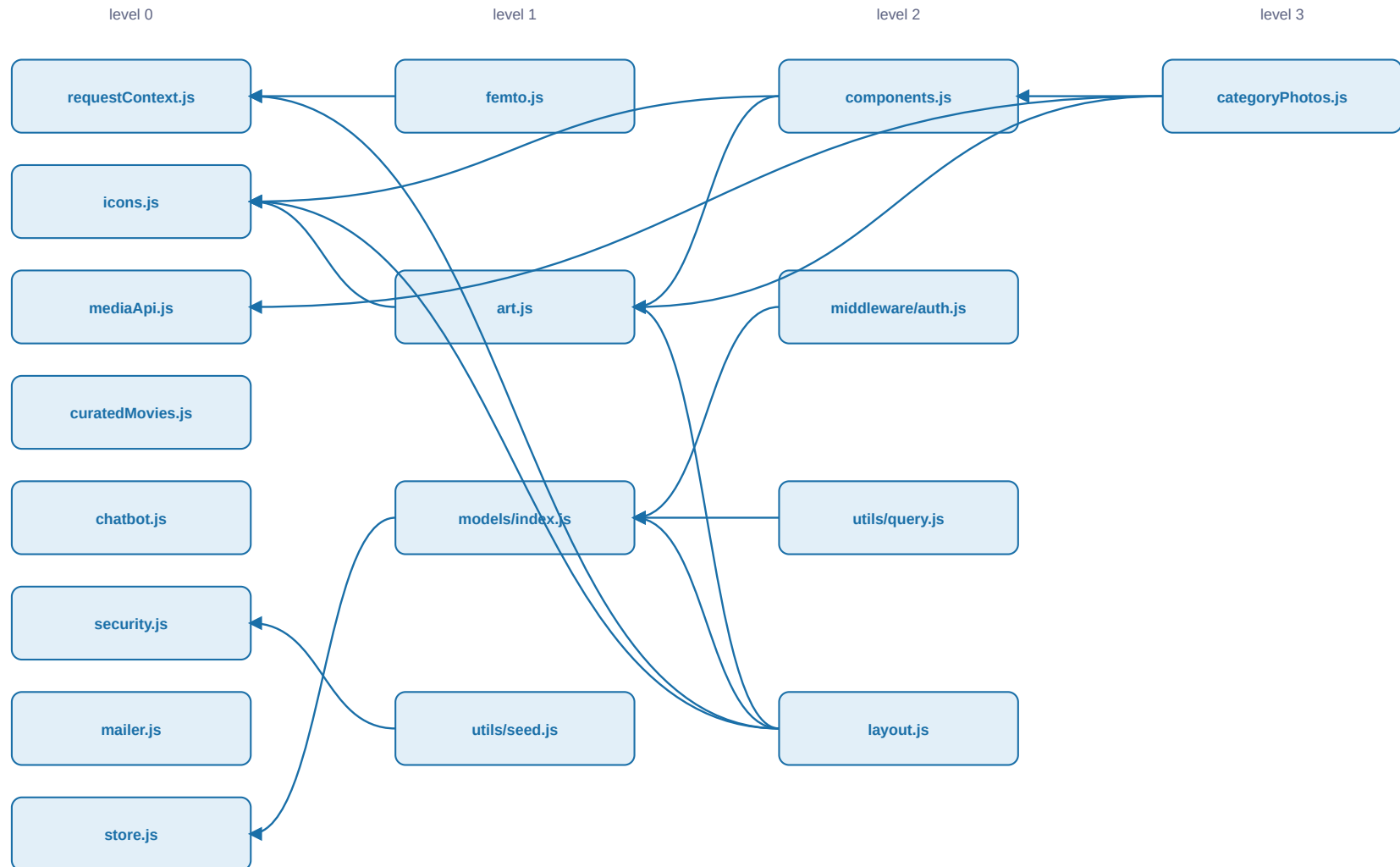
Every routes/*.js file on the left, and the exact views/*.js file(s) it renders on the right.



Orange arrows = a view file shared by more than one route (errors/404.js, dashboard/profile.js)

3 · core/ internals — who requires whom

The low-level building blocks (core/, middleware/, models/, utils/), laid out left → right by dependency depth.



Each column requires only files in columns to its left · level 0 = no internal dependencies

4 · Full file reference

Every file in the project, grouped by folder, with a one-line description and what it connects to.

Entry point

File	What it does	Connects to
<code>server.js</code>	Entry point. Creates the app, wires every middleware and every routes/*.js file, starts the HTTP server.	<code>core/femto.js</code> , <code>middleware/auth.js</code> , <code>routes/admin.js</code> , <code>routes/api.js</code> , <code>routes/articles.js</code> , <code>routes/auth.js</code> , <code>routes/characters.js</code> , <code>routes/content.js</code> , <code>routes/dashboard.js</code> , <code>routes/events.js</code> , <code>routes/media.js</code> , <code>routes/merch.js</code> , <code>routes/pages.js</code> , <code>routes/seo.js</code>

core/ — the framework & shared building blocks

File	What it does	Connects to
<code>core/art.js</code>	Generates deterministic placeholder art (gradient banners, avatar initials) from a text seed — no external image needed.	<code>core/icons.js</code>
<code>core/categoryPhotos.js</code>	Picks a photo for a category/content item, backed by mediaApi's fetch+cache; falls back to art.js when nothing else fits.	<code>core/art.js</code> , <code>core/components.js</code> , <code>core/mediaApi.js</code>
<code>core/chatbot.js</code>	Matches a visitor's message against the FAQ list (plus greetings/thanks) and returns a reply.	—
<code>core/components.js</code>	Shared HTML snippet builders reused across views: content cards, character cards, merch cards, event rows, star ratings, empty states.	<code>core/art.js</code> , <code>core/icons.js</code>
<code>core/curatedMovies.js</code>	Static list of the fan-made "Now Playing" movie rail entries shown on the homepage.	—
<code>core/femto.js</code>	The hand-rolled web framework itself: HTTP server, URL router, cookies & sessions, body parsing, static file serving, and res.render().	<code>core/requestContext.js</code>
<code>core/icons.js</code>	Inline-SVG icon set; exposes icon(name) used throughout views and components.	—
<code>core/layout.js</code>	The page shell every view is wrapped in: <head>, nav bar, footer, theme/font tags, and the <link>/<script> tags that load style.css and the front-end JS.	<code>core/art.js</code> , <code>core/icons.js</code> , <code>core/requestContext.js</code> , <code>models/index.js</code> , <code>public/css/style.css</code> , <code>public/js/chatbot.js</code> , <code>public/js/main.js</code>
<code>core/mailer.js</code>	Sends transactional email (verification codes, password resets) through the Brevo HTTP API.	—
<code>core/mediaApi.js</code>	Fetches photos/media from external providers (Unsplash / Pexels / OMDb) with an in-memory TTL cache.	—
<code>core/requestContext.js</code>	Tiny AsyncLocalStorage wrapper so any file can read the current request's protocol/host/url without it being passed down manually.	—
<code>core/security.js</code>	Password hashing & verification (scrypt), token/6-digit-code generation, and password-policy checking.	—
<code>core/store.js</code>	The whole "database": loads data/db.json into memory at startup, exposes collection(name) with find/insert/update/remove, debounced writes back to disk.	—

middleware/

File	What it does	Connects to
<code>middleware/auth.js</code>	Runs on every request: loads the signed-in user onto req.user/res.locals; exposes requireAuth / requireAdmin / redirectIfAuthenticated guards.	<code>models/index.js</code>

models/

File	What it does	Connects to
models/index.js	One named collection per data type (Users, Content, Characters, Merch, Events, Bookmarks, Feedback, ChatbotFaq, ChatbotLogs) — all thin wrappers over store.js.	core/store.js

utils/

File	What it does	Connects to
utils/query.js	Shared list helpers used by routes: attach a category to a record, sort by popular/A-Z/newest, look up a user's bookmarked ids, average a rating.	models/index.js
utils/seed.js	One-off script that fills data/db.json with demo users, content, characters, merch, events and FAQ entries.	core/security.js

routes/ — one file per area of the site, wired in server.js

File	What it does	Connects to
routes/admin.js	The whole admin dashboard: overview stats, and moderation screens for content, characters, merch, events, chatbot FAQ, feedback and users.	middleware/auth.js, models/index.js, utils/query.js, views/admin/charactersList.js, views/admin/chatbotFaq.js, views/admin/contentList.js, views/admin/eventsList.js, views/admin/feedbackList.js, views/admin/merchList.js, views/admin/overview.js, views/admin/usersList.js
routes/api.js	Small JSON endpoints (e.g. the chatbot reply) called by front-end JS.	core/chatbot.js, middleware/auth.js, models/index.js, utils/query.js
routes/articles.js	Article hub (reading) and the submit-an-article form.	core/categoryPhotos.js, middleware/auth.js, models/index.js, utils/query.js, views/articles/hub.js, views/articles/submit.js
routes/auth.js	Register, login (email verification code), logout, forgot/reset password.	core/mailer.js, core/requestContext.js, core/security.js, middleware/auth.js, models/index.js, views/auth/forgotPassword.js, views/auth/login.js, views/auth/loginVerify.js, views/auth/register.js, views/auth/resetPassword.js, views/dashboard/profile.js
routes/characters.js	Character list and character detail pages.	core/categoryPhotos.js, models/index.js, utils/query.js, views/characters/detail.js, views/characters/list.js, views/errors/404.js
routes/content.js	Explore/browse grid and the content detail page (articles, video, audio, image).	core/categoryPhotos.js, core/curatedMovies.js, core/mediaApi.js, models/index.js, utils/query.js, views/content/detail.js, views/content/explore.js, views/errors/404.js
routes/dashboard.js	The signed-in user's dashboard and profile settings.	core/categoryPhotos.js, middleware/auth.js, models/index.js, utils/query.js, views/dashboard/dashboard.js
routes/events.js	The events list / map page.	models/index.js, views/events/list.js
routes/media.js	Looks up media (photos) on demand for the front end via mediaApi.	core/mediaApi.js
routes/merch.js	Merch list and merch detail pages.	core/categoryPhotos.js, models/index.js, utils/query.js, views/errors/404.js, views/merch/detail.js, views/merch/list.js
routes/pages.js	Homepage, sitemap page and the feedback page.	core/categoryPhotos.js, core/curatedMovies.js, core/mediaApi.js, middleware/auth.js, models/index.js, utils/query.js, views/feedback.js, views/home.js, views/sitemap.js

File	What it does	Connects to
routes/seo.js	Generates /robots.txt and /sitemap.xml for search engines.	models/index.js

views/ — one file per page, called by routes via res.render()

File	What it does	Connects to
views/admin/charactersList.js	Admin: manage character profiles.	core/layout.js, views/admin/shell.js
views/admin/chatbotFaq.js	Admin: manage the chatbot's FAQ entries and view its chat logs.	core/layout.js, views/admin/shell.js
views/admin/contentList.js	Admin: moderate/manage articles, video, audio, image content.	core/layout.js, views/admin/shell.js
views/admin/eventsList.js	Admin: manage events.	core/layout.js, views/admin/shell.js
views/admin/feedbackList.js	Admin: review feedback submitted by users.	core/layout.js, views/admin/shell.js
views/admin/merchList.js	Admin: manage merch listings.	core/layout.js, views/admin/shell.js
views/admin/overview.js	Admin dashboard home: at-a-glance stats.	core/layout.js, views/admin/shell.js
views/admin/shell.js	Shared admin-panel frame (side nav + tabs) that every admin/*.js page renders inside of.	core/icons.js
views/admin/usersList.js	Admin: manage registered users.	core/layout.js, views/admin/shell.js
views/articles/hub.js	Public article-reading hub.	core/categoryPhotos.js, core/components.js, core/layout.js
views/articles/submit.js	"Submit an article" form.	core/layout.js
views/auth/forgotPassword.js	"Forgot password" request form.	core/layout.js
views/auth/login.js	Log-in form.	core/layout.js
views/auth/loginVerify.js	Enter-the-6-digit-code step after login.	core/layout.js
views/auth/register.js	Create-account form.	core/layout.js
views/auth/resetPassword.js	Set-a-new-password form (from the emailed reset link).	core/layout.js
views/characters/detail.js	One character's full profile page.	core/art.js, core/categoryPhotos.js, core/components.js, core/layout.js
views/characters/list.js	Browsable grid of characters.	core/categoryPhotos.js, core/components.js, core/layout.js
views/content/detail.js	One content item's detail page (article/video/audio/image).	core/art.js, core/categoryPhotos.js, core/components.js, core/layout.js
views/content/explore.js	The main explore/browse grid with filters and sorting.	core/categoryPhotos.js, core/components.js, core/layout.js
views/dashboard/dashboard.js	The signed-in user's dashboard (bookmarks, activity).	core/categoryPhotos.js, core/components.js, core/layout.js
views/dashboard/profile.js	Account/profile settings page.	core/layout.js
views/errors/403.js	"Forbidden" error page (non-admin hitting an admin route).	core/layout.js
views/errors/404.js	"Not found" error page.	core/layout.js
views/events/list.js	Events list with the interactive map (loads events-map.js).	core/components.js, core/layout.js, public/js/events-map.js
views/feedback.js	The feedback form page.	core/layout.js
views/home.js	Homepage: hero, browse-by-fandom grid, trending grid, now-playing rail, on-tour concert gallery + audio player, featured characters, upcoming events.	core/art.js, core/categoryPhotos.js, core/components.js, core/layout.js

File	What it does	Connects to
<code>views/merch/detail.js</code>	One merch item's detail page.	<code>core/art.js</code> , <code>core/categoryPhotos.js</code> , <code>core/components.js</code> , <code>core/layout.js</code>
<code>views/merch/list.js</code>	Merch grid.	<code>core/categoryPhotos.js</code> , <code>core/components.js</code> , <code>core/layout.js</code>
<code>views/sitemap.js</code>	The human-readable /sitemap page (also linked from the footer on every page).	<code>core/layout.js</code>

public/ — served as-is to the browser

File	What it does	Connects to
<code>public/css/style.css</code>	All site styling: layout, theme (light/dark) variables, components, animations.	loaded by every page via <code>core/layout.js</code>
<code>public/img/og-image.png</code>	Social-share preview image, referenced from <code>core/layout.js</code> 's meta tags.	referenced by <code>core/layout.js</code> 's social-preview meta tags
<code>public/js/chatbot.js</code>	Front-end chat widget; posts the visitor's message to <code>routes/api.js</code> and shows the reply.	loaded by every page via <code>core/layout.js</code> ; talks to <code>routes/api.js</code>
<code>public/js/events-map.js</code>	Front-end map/filtering behavior for the events list page.	loaded only by <code>views/events/list.js</code>
<code>public/js/main.js</code>	Front-end behavior for every page: nav, theme toggle, reveal-on-scroll, bookmarking, the homepage audio player, dropdowns.	loaded by every page via <code>core/layout.js</code>

database/

File	What it does	Connects to
<code>database/schema.sql</code>	Reference-only relational schema documenting the data model on paper. Not read by the running app — the real store is <code>store.js</code> + <code>data/db.json</code> .	documentation only — no code reads this file

data/

File	What it does	Connects to
<code>data/db.json</code>	The live data file — every user, article, character, merch item, event, bookmark and FAQ, as one JSON document.	read & written by <code>core/store.js</code>

Project config

File	What it does	Connects to
<code>.env.example</code>	Template listing the environment variables the app expects (API keys, mailer sender, etc.).	read by Node's <code>process.env</code> at startup (<code>server.js</code> / <code>core/mailer.js</code> / <code>core/mediaApi.js</code>)
<code>.gitignore</code>	Files Git should not track.	—
<code>README.md</code>	Project write-up / setup notes.	—
<code>env.txt</code>	The real environment values used on this machine (Brevo, Unsplash, Pexels, OMDb keys) — server-side only, never sent to the browser.	read by Node's <code>process.env</code> at startup (<code>server.js</code> / <code>core/mailer.js</code> / <code>core/mediaApi.js</code>)
<code>package.json</code>	Project metadata (name, start script). No npm dependencies — the whole stack is hand-rolled.	read by Node/npm to start the app ("npm start")